A CLASH ROYALE MACHINE LEARNING PROJECT

Leverage

The advantage you build before the first card is played

“Turn 33 million battles into your unfair advantage.”

Eight cards. One decision before the match begins. Leverage turns millions of real battles into a way to compare decks, strengthen a lineup, and build a response to an opponent. It is a project about making preparation measurable.

35,868,910 processed matches	**102** cards modeled	**61.88%** evaluation accuracy

Spandan Mukesh Deshpande

Originally developed as a student project at Manipal Institute of Technology. This report presents the idea, the machine learning system, the engineering, and the desktop application for a public GitHub audience.

01 Why the project is called Leverage

Clash Royale matches are won by people, but every player enters with a commitment to eight cards already made. A deck determines which threats can be answered, which win conditions can be supported, how quickly a player can cycle, and what options exist when the opponent reveals a strategy. The match is dynamic. The deck is the preparation that makes those decisions possible.

Leverage begins with a simple conviction: a better choice before the match can give a player an edge once it begins. I wanted to measure that edge from the decks themselves and put it in a player's hands. The name captures the moment when careful preparation makes each decision during the game go further. Its tagline, "Turn 33 million battles into your unfair advantage," invites players to use evidence when choosing a lineup.

That belief shaped three practical questions. If two decks are about to meet, which one does the model favor? If I want to strengthen a deck, which card swap earns its place across many matchups? If I know the opponent's eight cards, what lineup should I bring? The desktop application turns one trained predictor into tools for exploring each of these decisions.

A focus on deck construction

Leverage centers the choice a player can make before entering a match: the eight cards in the deck. During dataset preparation it selects matchups with starting trophy differences of at most 400 and average card level differences of at most 1.5. In the desktop experience, the two compared decks are evaluated at the same trophy and level settings, so the player can explore how changing the decks changes the prediction. This makes deck choice the actionable variable in the interface.

The resulting application is built for players who ask what gives my deck leverage against this one? Its answers come from a learned model of historical matches, coupled to search procedures that explore card swaps.

02 From battle records to a deck matchup

The project began with the Clash Royale Season 18 ladder dataset on Kaggle, attributed to BwandoWando. The notebook reads ten CSV match files and retains **35,868,910** usable rows after its cleaning and matchup selection rules. I also compiled a card attribute table for the **102 cards represented in this dataset**, including cost, targeting, damage, mobility, range, troop count, and strategic roles.

Protecting the question being asked

Each raw match labels its two sides as winner and loser. Feeding those positions directly into a classifier would teach it the answer from the file layout. The pipeline instead uses a random assignment with a fixed seed to put the winner on either Player 1 or Player 2, and creates a binary label indicating which neutral side won. It selects fields known before the match and excludes outcome fields such as crowns and tower state after play. The trophy and average level matchup filters then focus the learning dataset on closer contests, where deck composition is a useful thing to investigate.

Describing two decks with 264 numbers

Feature block	Width	Information represented
Player 1 card presence	102	Which eight cards are in the first deck, encoded against sorted card IDs.
Player 2 card presence	102	The corresponding encoding of the second deck's eight cards.
Deck summaries for each player	10 + 10	Aggregate composition, rarity, elixir, and statistics known before the match.
Card attributes for each player	20 + 20	Sums of the attributes of the eight cards: targeting, splash, roles, range, cycle, and more.

The two card blocks record the presence of cards: a 1 means the card is present and a 0 means it is absent. The 20 card attributes are added across the eight cards of each deck, creating 20 signals describing each deck. One row therefore describes a matchup between complete decks. Its exact feature order is preserved in the desktop prediction engine.

03 The engineering behind 35.9 million examples

Leverage processed tens of millions of matches on a Windows laptop with 24 GB of RAM and a 6 GB NVIDIA RTX 4050. Its data pipeline divides the work into manageable stages while preserving the full scale of the training set.

- **Read in chunks.** The notebook reads bounded batches from each source CSV, applies the cleaning and matchup filter, and encodes only the retained rows.
- **Store sparse features.** Each deck vector has 102 positions and contains only eight entries for cards that are present. SciPy CSR storage avoids writing hundreds of zero values for every match.
- **Save reusable parts.** Processed features are saved as **.npz** matrices with corresponding **.npy** labels. The pipeline can resume from these parts without rereading the entire raw dataset.
- **Convert precision deliberately.** Numeric storage is converted to float32 part by part, reducing memory used by feature values during later processing.
- **Build the XGBoost matrix by iteration.** A custom **xgb.DataIter** hands three consolidated training groups to **QuantileDMatrix(max_bin=128)**. XGBoost constructs its compact training representation from those groups before boosting on the GPU.

The training matrix contains **33,367,417 rows**. A separate processed part of **2,501,493 rows** is reserved for the final evaluation. The iterator reads batches during matrix construction, and training uses XGBoost's prepared QuantileDMatrix.

Making a long training run durable

The full XGBoost run spanned about 208 minutes and targeted 5,000 boosting rounds. A training callback saved model checkpoints every 250 rounds. The checkpoint at **round 4,750** was loaded, evaluated against the held out part, and saved as the project's final model. Checkpointing made the result a durable artifact that the desktop application could load independently of the training notebook.

04 Choosing the model

The notebook compares three approaches on a sample of two million matches: Logistic Regression, Random Forest, and XGBoost. The linear model provides a direct baseline; trees can respond to combinations of cards and deck properties; boosted trees add successive corrections to the prediction. XGBoost performed best when the models were compared on the same sample, so I carried it into tuning and the full run.

Experiment	Training setup	Evaluation accuracy
Logistic Regression	2M sampled matches; random 80/20 split	57.20%
Random Forest	Same 2M split	57.78%
XGBoost	Same 2M split; 200 rounds, depth 6	59.56%
Tuned XGBoost	2M sample; 3,000 rounds, depth 10	61.09%
Final model at full scale	33,367,417 training matches; 4,750 rounds	61.88%

The final model uses depth 10, learning rate 0.01, row and column subsampling of 0.8, L1 regularization 0.5, L2 regularization 2.0, histogram trees, and GPU acceleration. On the reserved evaluation part of 2,501,493 rows, the saved checkpoint achieved **61.8768% accuracy**, **61.9082% precision**, **61.8547% recall**, and **61.8814% F1**. The notebook reports the measured figures, while the rounded 61.88% gives readers an easy headline.

How the model reads a matchup

The feature vector presents both decks side by side. A boosted tree can split on card presence, elixir, role counts, and attributes from either player, then add its contribution to the running score. Across thousands of boosting rounds, the ensemble learns patterns that depend on the interaction between the two lineups. The trained predictor can then score any pair of complete decks represented by the application’s library of 102 cards.

05 Turning a prediction into a tool

Leverage is a PySide6 desktop application. Its interface uses a dark background, blue to represent the player's deck, red for the opponent, and a card palette with slots for eight cards that accept dragging and dropping. The inference module in **shared.py** loads the card table and the saved XGBoost model and builds the feature row of 264 values that the model expects. The interface in **main.py** turns that probability into three workflows.

Tool	The player's action	What the system does
Predict Winner	Build two complete decks.	Encodes the matchup and estimates the first deck's win probability.
Deck Improvement	Select a card to replace.	Scores eligible swaps across 15 sample opponent decks, allows at most 1.0 additional elixir on average, and returns the best three candidates.
Counter Deck	Build an opponent's deck.	Searches for a responding deck by swapping one card at a time, with four random restarts, up to five passes, and a cap of 5.0 on average elixir.

Why the searches matter

The model supplies a score for a pair of decks. The application builds useful searches around that score: generate candidate decks, apply rules for deck construction, evaluate matchups, and keep the strongest option found. Deck Improvement batches candidate predictions for efficient scoring. Counter Deck uses repeated improvement from several starting points to explore an enormous space of possible combinations of eight cards.

This bridge between machine learning and product design is central to Leverage. A probability is interesting. A suggested card swap is a decision a player can act on. An entire counter deck is a strategy the player can inspect and adapt.

06 What Leverage stands for

Every competitive player knows the feeling that a match can be shaped before the first card is played. A deck has a rhythm, answers to certain threats, and combinations that create pressure. One swap can change the way it handles a matchup. Leverage takes that intuition seriously enough to test it across millions of battles.

The finished project joins three kinds of work in one student project: a clear competitive question, machine learning at scale on a laptop, and a desktop tool that lets a player explore the result. The work moves from raw match records to a precise feature representation, from an ensemble trained on the GPU to a durable model artifact, and from a model score to three interactive deck decisions.

Leverage is about making preparation count. It turns the historical record of more than 35 million selected matches into a way to ask better questions about the eight cards you choose. That is the advantage behind the name: use evidence to make a stronger decision before the game asks you to execute it.